

# Matrix Theory and Computation

## Part 1: From matrix notation to computational kernels

Original slides 1–29 | Detailed lecture notes

Detailed teaching and study notes

First-year postgraduate course

Lecturer: Huiwen Wu

---

Step-by-step calculations, algorithm derivations, teaching prompts,  
verified textbook exercises, and worked solutions.

Based on the supplied Chapter 1 slides and Golub and Van Loan,  
*Matrix Computations*, 4th edition.

## Contents

|                                                   |    |
|---------------------------------------------------|----|
| Teaching guide and learning goals                 | 2  |
| 1. Dimensions are part of the mathematics         | 3  |
| 2. Dot products and vector updates                | 5  |
| 3. Matrix-vector multiplication: rows and columns | 7  |
| 4. Outer products and low-rank updates            | 9  |
| 5. One matrix product, three algorithmic views    | 10 |
| 6. Arithmetic counts, scaling, and BLAS           | 12 |
| 7. Bridge to Part 2: structure and bandwidth      | 13 |
| 8. Textbook exercises: assignment                 | 14 |
| 9. Textbook exercises: worked solutions           | 15 |
| 10. Additional practice and laboratory            | 17 |
| References and exercise provenance                | 18 |

## Teaching guide and learning goals

**Audience:** first-year postgraduate students in matrix theory and computation, numerical analysis, and machine learning. These notes assume elementary algebra and vectors, but do not assume experience implementing matrix algorithms.

**Scope:** original slides 1–29, ending with the band-matrix definition. Part 2 begins at original slide 30. Slide numbers in these notes always refer to the **original 87-page PDF**.

The central question is: **How can one mathematical product lead to several correct algorithms with different computational behavior?** Students should leave able to determine dimensions, execute algorithms by hand, explain what an update overwrites, count arithmetic under a stated convention, and connect loop ordering to data access.

The scheduled route is two 45-minute periods, excluding a break. The explanations, solutions, and extensions provide more material than should be read aloud. Use worked examples for board work and assign the longer exercises afterward.

| Time      | Slides | Teaching and student work                               |
|-----------|--------|---------------------------------------------------------|
| 0–8 min   | 1–5    | Motivate matrix kernels; check dimensions and notation. |
| 8–16 min  | 6–7    | E01: compute entries; compare product types.            |
| 16–27 min | 8–12   | E02–E03: trace dot and AXPY; explain a gradient step.   |
| 27–39 min | 13–17  | E04–E05: row and column gaxpy; shape checks.            |
| 39–45 min | 18–19  | E06: outer product and a rank-one correction.           |
| 45–60 min | 20–22  | E07: derive three multiplication views.                 |
| 60–72 min | 23–24  | Trace loop orderings and explain data access.           |
| 72–81 min | 25–27  | E08: flop counts, scaling, and BLAS levels.             |
| 81–87 min | 28–29  | Introduce structure and bandwidth.                      |
| 87–90 min | Review | Exit questions and assignment instructions.             |

**Preparation.** Draw three empty output arrays on the board: a scalar accumulator, a vector, and a matrix. Return to them when explaining dot, gaxpy, and outer-product accumulation. Ask students to identify the output object before doing arithmetic.

**Notation.** Vectors are columns. Indices start at 1 unless stated otherwise.  $A(:, j)$  means column  $j$ ;  $A(i, :)$  means row  $i$ . The assignment symbol  $\leftarrow$  overwrites a variable. For  $A \in \mathbb{R}^{m \times r}$  and  $B \in \mathbb{R}^{r \times n}$ ,  $r$  is the shared or contracted dimension.

## 1. Dimensions are part of the mathematics

**Slides 3–7.** Begin with: “Before asking how to compute a product, establish what kind of object it produces.”

For  $A \in \mathbb{R}^{m \times r}$  and  $B \in \mathbb{R}^{r \times n}$ , each row of  $A$  and each column of  $B$  have  $r$  entries. Their dot product is defined. There are  $m$  rows and  $n$  columns, so

$$AB \in \mathbb{R}^{m \times n}, \quad (AB)_{ij} = \sum_{k=1}^r a_{ik}b_{kj}.$$

The shared index  $k$  disappears because we sum over it; the free indices  $i, j$  label the answer.

### E01: calculate every output entry

The slides use

$$A = \begin{bmatrix} 1 & 2 & 0 \\ -1 & 3 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 \\ 0 & -1 \\ 4 & 2 \end{bmatrix}.$$

Check  $(2 \times 3)(3 \times 2) \rightarrow 2 \times 2$ , then compute:

$$\begin{aligned} c_{11} &= 1(2) + 2(0) + 0(4) = 2, \\ c_{12} &= 1(1) + 2(-1) + 0(2) = -1, \\ c_{21} &= (-1)(2) + 3(0) + 1(4) = 2, \\ c_{22} &= (-1)(1) + 3(-1) + 1(2) = -2. \end{aligned}$$

Thus

$$AB = \begin{bmatrix} 2 & -1 \\ 2 & -2 \end{bmatrix}.$$

Have students point to the input row and column for  $c_{21}$  before calculating it. This distinguishes an indexing mistake from an arithmetic mistake.

|                                                                                                                                                                     |          |     |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----|-----|----------|---|---|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|----|---|---|--|--------------------------------------------------------------------------------------------------------------------------------------------------|---|----|---|----|
| $A$                                                                                                                                                                 | $\times$ | $B$ | $=$ | $C = AB$ |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
| <table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>1</td><td>2</td><td>0</td></tr> <tr><td>-1</td><td>3</td><td>1</td></tr> </table> | 1        | 2   | 0   | -1       | 3 | 1 |  | <table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>2</td><td>1</td></tr> <tr><td>0</td><td>-1</td></tr> <tr><td>4</td><td>2</td></tr> </table> | 2 | 1 | 0 | -1 | 4 | 2 |  | <table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>2</td><td>-1</td></tr> <tr><td>2</td><td>-2</td></tr> </table> | 2 | -1 | 2 | -2 |
| 1                                                                                                                                                                   | 2        | 0   |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
| -1                                                                                                                                                                  | 3        | 1   |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
| 2                                                                                                                                                                   | 1        |     |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
| 0                                                                                                                                                                   | -1       |     |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
| 4                                                                                                                                                                   | 2        |     |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
| 2                                                                                                                                                                   | -1       |     |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |
| 2                                                                                                                                                                   | -2       |     |     |          |   |   |  |                                                                                                                                                                               |   |   |   |    |   |   |  |                                                                                                                                                  |   |    |   |    |

Highlighted row and column meet at  $c_{12}$   
 $[1, 2, 0] [1, -1, 2]^T = 1 - 2 + 0 = -1$

**Try it:** Which row and column would you highlight to obtain  $c_{21}$ ?

**Figure 1:** Original slide 7: one row and one column determine an output entry.

The reverse product exists, but has a different size:

$$BA = \begin{bmatrix} 1 & 7 & 1 \\ 1 & -3 & -1 \\ 2 & 14 & 2 \end{bmatrix} \in \mathbb{R}^{3 \times 3}.$$

Its first row is  $2A(1, :) + A(2, :) = [1, 7, 1]$ . Writing  $AB = BA$  would compare objects in different matrix spaces.

### Three operations that must not be confused

| Operation                    | Requirement            | Output                        |
|------------------------------|------------------------|-------------------------------|
| Matrix product $AB$          | Shared inner dimension | Rows of $A$ by columns of $B$ |
| Hadamard product $A \circ B$ | Identical shapes       | Same shape as the inputs      |
| Outer product $xy^T$         | Lengths $m$ and $n$    | An $m \times n$ matrix        |

The Hadamard product is undefined for the E01 matrices because their shapes differ. For  $u = (1, 2)^T$  and  $v = (3, 4)^T$ ,

$$u^T v = 11, \quad u \circ v = \begin{bmatrix} 3 \\ 8 \end{bmatrix}, \quad uv^T = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix}.$$

These are a scalar, a vector, and a matrix. Ask students to name the output type without calculating entries.

**ML connection.** Store  $b$  samples with  $d$  features as rows of  $X \in \mathbb{R}^{b \times d}$ . If  $W \in \mathbb{R}^{d \times h}$ , the linear layer  $XW$  has shape  $b \times h$ . A bias of length  $h$  is added to each row. Broadcasting is an implementation convention, not a change to matrix-product dimensions.

**Check.** If  $A$  is  $4 \times 3$  and  $x$  has length 4,  $Ax$  is undefined.  $A^T x$  is defined and has length 3.

## 2. Dot products and vector updates

**Slides 8–12.** The difference between a mathematical expression and a state-changing algorithm is easiest to see with vectors.

### E02: the dot-product accumulator

For  $x = (1, -2, 3)^T$  and  $y = (4, 0, -1)^T$ ,

$$x^T y = 1(4) + (-2)(0) + 3(-1) = 1.$$

```
c = 0
for i = 1,...,n:
    c = c + x[i]*y[i]
return c
```

| Completed iteration | Product just formed | Value of $c$ |
|---------------------|---------------------|--------------|
| Initialization      | None                | 0            |
| $i = 1$             | $1(4) = 4$          | 4            |
| $i = 2$             | $(-2)(0) = 0$       | 4            |
| $i = 3$             | $3(-1) = -3$        | 1            |

**Why it works.** After iteration  $i$ ,

$$c = \sum_{j=1}^i x_j y_j.$$

This is a *loop invariant*. Initially the empty sum is zero. Each iteration appends one missing term. At  $i = n$ , it is the desired dot product. The same reasoning will prove matrix and block algorithms.

**Count.** The displayed loop performs  $n$  multiplications and  $n$  additions, including the first addition to zero:  $2n$  flops. Initializing  $c = x_1 y_1$  and starting at 2 costs  $2n - 1$ . Both are  $O(n)$ .

Do not remove the zero multiplication in E02 when counting the displayed generic loop. A program that exploits known zeros is a different algorithm.

**Complex extension.** Use  $x^* y = \sum_j \overline{x_j} y_j$  for a complex inner product. For  $x = (1, i)^T$ ,  $x^T x = 0$  but  $x^* x = 2$ . Transposition and conjugate transposition differ.

### E03: AXPY changes a vector

AXPY replaces  $y$  by  $y + ax$ :

$$y_{\text{new}} = y_{\text{old}} + ax.$$

With  $a = 2$ ,  $x = (1, -2, 3)^T$ , and  $y_{\text{old}} = (4, 0, -1)^T$ ,

$$y_{\text{new}} = \begin{bmatrix} 4 + 2(1) \\ 0 + 2(-2) \\ -1 + 2(3) \end{bmatrix} = \begin{bmatrix} 6 \\ -4 \\ 5 \end{bmatrix}.$$

```
for i = 1,...,n:
    y[i] = y[i] + a*x[i]
```

The input  $x$  and scalar  $a$  stay fixed. Each output position is updated once. The work is  $2n$  flops, with constant extra scalar workspace.

In BLAS, SAXPY is the single-precision real routine and DAXPY is the double-precision real routine. They implement the same update.

### A fully specified gradient-descent example

Take

$$w = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad g = \begin{bmatrix} 2 \\ -3 \end{bmatrix}, \quad \eta = 0.1.$$

Then

$$w_{\text{new}} = w - \eta g = \begin{bmatrix} 1 - 0.2 \\ -1 + 0.3 \end{bmatrix} = \begin{bmatrix} 0.8 \\ -0.7 \end{bmatrix}.$$

The AXPY arguments are  $a = -\eta$ ,  $x = g$ , and  $y = w$ . The displacement  $(-0.2, 0.3)^T$  points opposite to the gradient.

Define

$$f(w) = \frac{1}{2} w^T H w, \quad H = \text{diag}(2, 3).$$

At  $w = (1, -1)^T$ ,  $\nabla f(w) = Hw = (2, -3)^T$ . The objective decreases from  $f(w) = 2.5$  to

$$f(w_{\text{new}}) = \frac{1}{2} \{2(0.8)^2 + 3(-0.7)^2\} = 1.375.$$

The decrease depends on the objective and step size; it is not guaranteed by AXPY alone. Each coordinate is multiplied by  $1 - \eta\lambda_i$ . Convergence here requires  $|1 - \eta\lambda_i| < 1$ , satisfied for  $0 < \eta < 2/3$ .

**Prompt.** If the old iterate is needed later, save it before overwriting  $w$ .

### 3. Matrix-vector multiplication: rows and columns

**Slides 13–17.** Gaxpy computes  $y \leftarrow y + Ax$ . The initial  $y$  is part of the input.

#### Two derivations

Write  $A$  by rows,  $A = [r_1, \dots, r_m]^T$ . Each output is a dot-product update:

$$y_i \leftarrow y_i + r_i^T x = y_i + \sum_{j=1}^n a_{ij} x_j.$$

Write  $A = [c_1, \dots, c_n]$  by columns instead:

$$Ax = \sum_{j=1}^n x_j c_j.$$

The row view finishes one output coordinate at a time. The column view adds the contribution of one input coordinate to all outputs.

Row-oriented:

```
for i = 1,...,m:
    for j = 1,...,n:
        y[i] = y[i] + A[i,j]*x[j]
```

Column-oriented:

```
for j = 1,...,n:
    for i = 1,...,m:
        y[i] = y[i] + A[i,j]*x[j]
```

Each performs  $mn$  multiply-add updates:  $2mn$  flops. The column-loop invariant is

$$y^{(j)} = y^{(0)} + \sum_{\ell=1}^j A(:, \ell) x_\ell.$$

#### E04: full numerical trace

Use

$$A = \begin{bmatrix} 1 & 2 \\ -1 & 3 \\ 2 & 0 \end{bmatrix}, \quad x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad y^{(0)} = \begin{bmatrix} 1 \\ 0 \\ -2 \end{bmatrix}.$$

The row calculation is

$$\begin{aligned} (Ax)_1 &= 1(2) + 2(-1) = 0, \\ (Ax)_2 &= (-1)(2) + 3(-1) = -5, \\ (Ax)_3 &= 2(2) + 0(-1) = 4. \end{aligned}$$

Therefore  $y^{(0)} + Ax = (1, -5, 2)^T$ .

The column calculation gives intermediate states:

$$y^{(1)} = \begin{bmatrix} 1 \\ 0 \\ -2 \end{bmatrix} + 2 \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ -2 \\ 2 \end{bmatrix},$$

$$y^{(2)} = \begin{bmatrix} 3 \\ -2 \\ 2 \end{bmatrix} - \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -5 \\ 2 \end{bmatrix}.$$

With an initial zero vector the result would be  $(0, -5, 4)^T$ . Changing initialization changes the problem.

**Aliasing pitfall.** Reading  $x$  must be unaffected by overwriting  $y$ . Using the same storage can change an input needed later. For

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad x = (1, 2)^T,$$

an unsafe attempt to overwrite  $x$  with  $Ax$  can set  $x_1 = 3$  and then compute  $x_2 = 3 + 2 = 5$ , instead of 3. Use a separate output or an input copy.

### E05: slices have shapes

Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}, \quad x = (2, -1)^T.$$

$A(2, :) = [3, 4]$  is a  $1 \times 2$  row, and

$$A(2, :)x = 3(2) + 4(-1) = 2$$

is a scalar.  $A(:, 2) = (2, 4, 6)^T$  is a  $3 \times 1$  column, and

$$A(:, 2)x_2 = (-2, -4, -6)^T$$

is a vector. In the second expression  $x_2$  is a scalar.

**Board exercise.** Compute  $Ax = (0, 2, 4)^T$  by both views. Each row dot product has length 2; the column formulation adds two length-3 vectors.

**ML connection.** For a prediction vector  $Xw$ , feature  $j$  contributes  $w_j X(:, j)$  to all predictions. The row view computes the prediction for one observation.

## 4. Outer products and low-rank updates

**Slides 18–19.** The vector lengths in an outer product need not match:

$$x \in \mathbb{R}^m, \quad y \in \mathbb{R}^n \quad \implies \quad xy^T \in \mathbb{R}^{m \times n}.$$

Its  $(i, j)$  entry is  $x_i y_j$ ; its  $j$ th column is  $y_j x$ .

### E06: build and add the correction

For

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = \begin{bmatrix} 3 \\ -1 \\ 2 \end{bmatrix},$$

construct one column at a time:

$$xy^T = [3x, -x, 2x] = \begin{bmatrix} 3 & -1 & 2 \\ 6 & -2 & 4 \end{bmatrix}.$$

With

$$A_0 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix},$$

the update gives

$$A_0 + xy^T = \begin{bmatrix} 4 & -1 & 3 \\ 6 & -1 & 4 \end{bmatrix}.$$

The first two columns have determinant  $4(-1) - (-1)6 = 2$ . The updated matrix has rank 2 although the correction has rank 1.

### Why the correction has rank one

Every output of the map  $z \mapsto xy^T z$  is

$$(xy^T)z = x(y^T z),$$

a scalar multiple of  $x$ . Its image lies in  $\text{span}\{x\}$ . If both vectors are nonzero, some  $z$  satisfies  $y^T z \neq 0$ , so the image is exactly one-dimensional. If either vector is zero, the rank is zero.

This gives an efficient application algorithm:

```
s = dot(y,z)
w = s*x
```

It costs  $O(m+n)$  and stores the factors in  $m+n$  numbers, instead of forming  $mn$  entries. For  $z = (1, 0, -1)^T$ ,  $y^T z = 1$ , so  $(xy^T)z = (1, 2)^T$ .

**Learning connection.** A correction  $\Delta W = uv^T$  extracts one scalar feature  $v^T z$  and distributes it in direction  $u$ . A sum of several such corrections can represent a higher-rank change.

**Prompt.** A rank-one correction does not mean that the corrected model produces only one-dimensional outputs. It describes the correction, not the pre-existing matrix.

## 5. One matrix product, three algorithmic views

Slides 20–24. Reuse

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

These matrices return in the Strassen and Cannon examples in Part 2.

### E07: dot, row update, and outer-product accumulation

The dot view computes individual entries:

$$AB = \begin{bmatrix} 1(5) + 2(7) & 1(6) + 2(8) \\ 3(5) + 4(7) & 3(6) + 4(8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

The row-update view computes, for example,

$$C(1, :) = [0, 0] + 1[5, 6] + 2[7, 8] = [19, 22].$$

The outer-product view computes

$$C^{(1)} = A(:, 1)B(1, :) = \begin{bmatrix} 5 & 6 \\ 15 & 18 \end{bmatrix},$$

$$C^{(2)} = C^{(1)} + A(:, 2)B(2, :) = \begin{bmatrix} 5 & 6 \\ 15 & 18 \end{bmatrix} + \begin{bmatrix} 14 & 16 \\ 28 & 32 \end{bmatrix}.$$

Both summands have rank 1. Their sum has determinant  $19(50) - 22(43) = 4$ , hence rank 2.

$$\begin{array}{ccc} A_{:,1}B_{1,:} & & A_{:,2}B_{2,:} \\ \begin{array}{|c|c|} \hline 5 & 6 \\ \hline 15 & 18 \\ \hline \end{array} & + & \begin{array}{|c|c|} \hline 14 & 16 \\ \hline 28 & 32 \\ \hline \end{array} \\ & & = \end{array} \quad \begin{array}{|c|c|} \hline 19 & 22 \\ \hline 43 & 50 \\ \hline \end{array} \quad AB$$

Each outer product fills the whole matrix.  
Adding both rank-one contributions gives the final product.

**Try it:** Why can each layer have rank one while their sum has rank two?

**Figure 2:** Original slide 22: two rank-one contributions sum to the product.

### Derive the outer-product identity

For compatible matrices,

$$AB = \sum_{k=1}^r A(:, k)B(k, :).$$

Check entry  $(i, j)$ :

$$\left[ \sum_{k=1}^r A(:, k) B(k, :) \right]_{ij} = \sum_{k=1}^r a_{ik} b_{kj} = (AB)_{ij}.$$

The corresponding loop invariant is

$$C^{(k)} = C^{(0)} + \sum_{\ell=1}^k A(:, \ell) B(\ell, :).$$

It explains both correctness and the meaning of a partial computation.

## Six loop orderings

The scalar update is

$$C(i, j) \leftarrow C(i, j) + A(i, k) B(k, j).$$

Permuting the loops gives:

| Loop order     | Innermost action | Inner-loop data            |
|----------------|------------------|----------------------------|
| $ijk$ or $jik$ | Dot accumulation | Row of $A$ , column of $B$ |
| $ikj$ or $kij$ | Row AXPY         | Rows of $B$ and $C$        |
| $jki$ or $kji$ | Column AXPY      | Columns of $A$ and $C$     |

In  $ijk$ , the products for  $c_{11}$ ,  $1(5)$  and  $2(7)$ , occur consecutively. In  $kij$ , the first outer-product layer is completed before the second.

In column-major storage, varying  $i$  traverses consecutive entries. The inner  $i$  loop in  $jki$  accesses  $A(:, k)$  and  $C(:, j)$  contiguously while reusing  $B(k, j)$ . Row-major storage favors different patterns. Runtime also depends on blocking, vectorization, sizes, and implementation.

## Numerical equality needs a qualification

All six scalar loop nestings, with  $k$  increasing, add terms of each individual  $c_{ij}$  in the same  $k$  order. Merely permuting these loops need not alter its arithmetic order. But vectorized reductions, parallel summation, blocking, and fused multiply-add instructions can alter rounding.

In a simplified three-significant-digit decimal system,  $(1000+1) - 1000$  can evaluate to zero because the first sum rounds to 1000. However,  $1000 + (1 - 1000) = 1$ . Real-number associativity is not a guarantee about floating-point evaluation.

**Check.** Initialize  $C$  to zero for  $C = AB$ . Keep its old value for  $C \leftarrow C + AB$ .

## 6. Arithmetic counts, scaling, and BLAS

**Slides 25–27.** A multiply-add update counts as two flops. A hardware instruction performing both is conventionally counted as two, even if it executes as one instruction.

### E08: rectangular multiplication

For a  $3 \times 2$  matrix times a  $2 \times 4$  matrix, there are 12 output entries, each with 2 updates:

$$12 \times 2 = 24 \text{ updates} = 48 \text{ flops.}$$

If each entry is directly formed from two products and one addition, the count is  $12(2 + 1) = 36$  flops. State which implementation is counted.

| Kernel                            | Generic update count | Complexity |
|-----------------------------------|----------------------|------------|
| Dot product                       | $2n$                 | $O(n)$     |
| AXPY                              | $2n$                 | $O(n)$     |
| Gaxpy, $m \times n$ matrix        | $2mn$                | $O(mn)$    |
| Outer-product update              | $2mn$                | $O(mn)$    |
| Product with dimensions $m, r, n$ | $2mrn$               | $O(mrn)$   |

For square dense multiplication, doubling  $n$  multiplies leading work by 8 and storage by 4. Runtime need not change by exactly those factors.

**A hypothetical estimate.** At an assumed sustained rate of 100 GFLOP/s, the arithmetic-only estimate for  $n = 1000$  is

$$\frac{2(1000)^3}{100 \times 10^9} = 0.02 \text{ s.}$$

This is not a benchmark. Allocation, data transfers, execution overhead, and changing throughput are omitted.

### Why Level 3 matters

Level 1 BLAS mainly operates on vectors. Level 2 uses matrix-vector operations. In both cases, work and data grow at comparable rates. Level 3 performs matrix-matrix operations and can reuse the same data for substantially more arithmetic.

An ideal square product that reads  $A, B$  once and writes  $C$  once in 8-byte arithmetic transfers approximately  $24n^2$  bytes. Its ideal arithmetic intensity is

$$\frac{2n^3}{24n^2} = \frac{n}{12} \text{ flops per byte.}$$

This models ideal reuse, not an assertion that the whole product fits in cache. Tiling seeks useful reuse on smaller blocks.

A matrix-vector product reads about  $8n^2$  matrix bytes for  $2n^2$  leading flops: approximately 1/4 flop per byte. This explains why the two kernels can face different hardware bottlenecks.

**Application.** Combining input vectors as columns of  $X$  turns repeated  $Ax_j$  computations into  $AX$ . The work is similar, but a blocked implementation can reuse  $A$  across several inputs.

## 7. Bridge to Part 2: structure and bandwidth

**Slides 28–29.** Dense algorithms process all index combinations. Known structure can avoid unnecessary work.

Lower bandwidth at most  $p$  and upper bandwidth at most  $q$  mean

$$a_{ij} = 0 \quad \text{if } i - j > p \text{ or } j - i > q.$$

Potentially nonzero entries in row  $i$  satisfy

$$\max(1, i - p) \leq j \leq \min(n, i + q).$$

In column  $j$ , they satisfy

$$\max(1, j - q) \leq i \leq \min(m, j + p).$$

Derive the bounds rather than memorize them: students often swap  $p$  and  $q$  in the column form.

For the slide's  $8 \times 5$  example,  $p = 1$ ,  $q = 2$ . Column 3 allows rows 1 through 4. Row 6 permits only column 5. Rows 7 and 8 are zero. Rectangular band matrices can have entire zero rows.

A tridiagonal  $n \times n$  matrix has  $p = q = 1$  and at most

$$n + (n - 1) + (n - 1) = 3n - 2$$

potentially nonzero entries. Part 2 starts with a second-difference operator having exactly that structure.

### Exit questions.

1. What distinguishes  $C = AB$  from  $C \leftarrow C + AB$ ?
2. Why can two rank-one matrices sum to a rank-two matrix?
3. Why does a flop count not determine runtime?
4. With  $p = 2$ ,  $q = 1$ , which column indices may occur in row 5?

**Answers.** Zero versus existing-output initialization; different column spaces; data movement and overhead also matter; columns 3 through 6, clipped to the actual column range.

## 8. Textbook exercises: assignment

These are **paraphrased from verified problems** in Golub and Van Loan, *Matrix Computations*, 4th edition, Section 1.1, printed page 13. Numerical instances and teaching prompts are added here. Solutions are newly worked for these notes.

Assign T1 and T3 as core work; use T2 in a tutorial and T4 for further algorithm-design practice.

### T1. Matrix-chain order — Problem P1.1.4

For  $A \in \mathbb{R}^{m \times n}$ ,  $B \in \mathbb{R}^{n \times p}$ ,  $C \in \mathbb{R}^{p \times q}$ , compare the leading costs of  $(AB)C$  and  $A(BC)$ . When is the first cheaper?

**Added instance.** Use dimensions  $100 \times 5$ ,  $5 \times 100$ ,  $100 \times 5$ . Report intermediate sizes and flop counts.

### T2. Trace all loop orders — Problem P1.1.2

For conventional  $2 \times 2$  multiplication, list the eight multiplication terms in the order executed by each of the six loop nestings. Let all indices increase from 1 to 2.

**Added prompt.** Use E07. Which order groups terms into full outer-product layers?

### T3. Powers of a rank-one matrix — Problem P1.1.3

Develop an efficient procedure for  $(xy^T)^k$ , where  $x, y \in \mathbb{R}^n$  and  $k$  is a positive integer. Explain the  $O(n^2)$  cost for fixed  $k$ .

**Added instance.** Take  $x = (1, 2)^T$ ,  $y = (3, -1)^T$ ,  $k = 4$ . Discuss the separate case  $y^T x = 0$ .

### T4. Compute only one column — Problem P1.1.1

Let

$$M = (A - \xi_1 I)(A - \xi_2 I) \cdots (A - \xi_r I), \quad A \in \mathbb{R}^{n \times n}.$$

Return the first column without forming all of  $M$ .

**Added instance.** Use

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}, \quad \xi_1 = 1, \quad \xi_2 = 2.$$

## 9. Textbook exercises: worked solutions

### T1 solution: intermediate dimensions control the work

For  $(AB)C$ , an  $m \times p$  intermediate costs  $2mnp$ , followed by  $2mpq$ :

$$W_{\text{left}} = 2mp(n + q).$$

For  $A(BC)$ , first form an  $n \times q$  intermediate:

$$W_{\text{right}} = 2npq + 2mnq = 2nq(p + m).$$

The left ordering is cheaper when

$$mp(n + q) < nq(p + m).$$

For the added dimensions,

$$W_{\text{left}} = 100,000 + 100,000 = 200,000, \quad W_{\text{right}} = 5,000 + 5,000 = 10,000.$$

The intermediates are  $100 \times 100$  and  $5 \times 5$ . The right ordering uses 20 times less leading arithmetic. Associativity permits changing parentheses, not rearranging factors.

### T2 solution: explicit traces

Write  $t_{ijk} = a_{ik}b_{kj}$ . Its indices record output row, output column, and shared index.

| Loop order | Ordered triples $(i, j, k)$            |
|------------|----------------------------------------|
| $ijk$      | 111, 112, 121, 122, 211, 212, 221, 222 |
| $jik$      | 111, 112, 211, 212, 121, 122, 221, 222 |
| $ikj$      | 111, 121, 112, 122, 211, 221, 212, 222 |
| $jki$      | 111, 211, 112, 212, 121, 221, 122, 222 |
| $kij$      | 111, 121, 211, 221, 112, 122, 212, 222 |
| $kji$      | 111, 211, 121, 221, 112, 212, 122, 222 |

For example,  $t_{111} = 5$ ,  $t_{121} = 6$ ,  $t_{211} = 15$ ,  $t_{221} = 18$ . These are the complete  $k = 1$  outer-product layer. Orders  $kij$  and  $kji$  keep each layer together.

Every output receives the same two products in every ordering. The set of terms verifies correctness; their ordering describes computational organization.

### T3 solution: reduce the matrix power to a scalar power

Let  $R = xy^T$  and  $s = y^T x$ . Then

$$R^2 = xy^T xy^T = x(y^T x)y^T = sR.$$

Induction gives

$$R^k = s^{k-1}xy^T, \quad k \geq 1.$$

Handle  $k = 1$  separately so the  $s = 0$  case does not depend on  $0^0$ .

```

if k == 1:
    return outer(x,y)
s = dot(y,x)
t = scalar_power(s,k-1)
return t*outer(x,y)

```

The dot costs  $O(n)$  and dense output formation costs  $O(n^2)$ . Exponentiation by squaring costs  $O(\log k)$  scalar operations. Total:  $O(n^2 + \log k)$ , or  $O(n^2)$  for fixed  $k$ .

Here  $s = 3 - 2 = 1$ , so

$$R^4 = R = \begin{bmatrix} 3 & -1 \\ 6 & -2 \end{bmatrix}.$$

If  $s = 0$ , then  $R^2 = 0$ , possibly with  $R \neq 0$ . For  $x = (1, 0)^T$ ,  $y = (0, 1)^T$ ,

$$R = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad R^2 = 0.$$

#### T4 solution: apply the product to a basis vector

The first column is  $Me_1$ . Apply the rightmost factor first:

```

v = e1
for j = r,r-1,...,1:
    w = A*v
    v = w - xi[j]*v
return v

```

Keep the old  $v$  while forming both  $Av$  and  $\xi_j v$ . Dense work is  $O(rn^2)$ , with  $O(n)$  auxiliary vector storage beyond  $A$ .

For the numerical instance,  $(A - 2I)e_1 = 0$ , so the final result is zero. Check:

$$(A - I)(A - 2I) = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ 0 & 2 \end{bmatrix}.$$

Its first column is zero. The instance is deliberately simple; the same procedure works with nonzero intermediate vectors.

## 10. Additional practice and laboratory

These are **instructor-created exercises**, not numbered textbook questions.

### P1. An update versus a fresh product

For

$$A = \begin{bmatrix} 2 & -1 \\ 1 & 3 \end{bmatrix}, \quad x = (1, 2)^T, \quad y_0 = (4, -1)^T,$$

compute both  $Ax$  and  $y_0 + Ax$ .

**Solution.**  $Ax = (0, 7)^T$  and  $y_0 + Ax = (4, 6)^T$ . The column trace is  $(4, -1)^T + (2, 1)^T = (6, 0)^T$ , then add  $2(-1, 3)^T$ .

### P2. Apply a low-rank matrix

For  $u = (1, -1, 2)^T$ ,  $v = (2, 0)^T$ ,  $z = (3, 4)^T$ , compute  $(uv^T)z$  without forming the matrix.

**Solution.**  $v^T z = 6$ , hence  $(6, -6, 12)^T$ . Only a dot and a scaling are needed.

### P3. Compare exact counts

For dimensions  $(2 \times 3)(3 \times 4)$ , give update and direct-sum counts.

**Solution.** Eight outputs, each with three terms:  $8(2 \cdot 3) = 48$  update flops or  $8(3 + 2) = 40$  direct-sum flops.

### Laboratory: reference implementations

Implement both gaxpy views and compare with a trusted product. These routines use one-dimensional NumPy vectors.

```
import numpy as np

def gaxpy_rows(A, x, y0):
    m, n = A.shape
    assert x.shape == (n,) and y0.shape == (m,)
    y = y0.astype(float, copy=True)
    for i in range(m):
        for j in range(n):
            y[i] += A[i, j] * x[j]
    return y

def gaxpy_columns(A, x, y0):
    m, n = A.shape
    assert x.shape == (n,) and y0.shape == (m,)
    y = y0.astype(float, copy=True)
    for j in range(n):
        for i in range(m):
            y[i] += A[i, j] * x[j]
    return y
```

Use E04 and P1 as checks; compare with  $y_0 + A @ x$  using a numerical tolerance. Timing these Python loops includes interpreter overhead and cannot establish the performance ordering of optimized BLAS kernels.

**Assessment.** Award credit for dimensions, initialization, a correct accumulation invariant, and a stated counting convention. A final matrix alone is incomplete evidence of algorithmic understanding.

## References and exercise provenance

1. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. Main reading: Sections 1.1–1.2. T1–T4 use Problems P1.1.4, P1.1.2, P1.1.3, and P1.1.1, printed page 13. The edition and exercise page were checked in the [textbook PDF in a computational-physics teaching repository](#).
2. M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Supplementary reading: Chapter 2. [Author-maintained book site](#).
3. R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2012.
4. 王磊, 《矩阵理论与计算》. Chinese-language course reference supplied by the instructor. Publisher, year, edition, and exercise numbers were not supplied; no numbered problem is attributed to this book without verification.

Figures are extracted from the instructor’s slides. Textbook statements are paraphrased; calculations, extensions, and solutions are prepared for this course.